



BigData

What is BigData

Big Data is not a programming language — it is a concept/problem domain involving data that is too large, fast, or complex for a single computer. Hadoop and Spark are not “other words” for Big Data; instead, they are technologies used to work with Big Data. Hadoop is an ecosystem (HDFS storage + YARN resource manager + MapReduce engine), while Apache Spark is a fast distributed data-processing framework that can run with or without Hadoop. Spark is mainly implemented in Scala, runs on the JVM, and can be used from Java. Spark SQL is a library on top of Spark that lets us run SQL queries on large distributed datasets.

- Big Data → **problem space**
- Hadoop → **ecosystem for storage + processing**
- Spark → **fast distributed processing framework**
- Spark is written in → **Scala (on JVM)**
- I use → **Spark SQL from Java** to write queries on big datasets

And that's **exactly right**.

Should i have to Learn Hadoop

You don't *have* to learn **Hadoop MapReduce**, but you should understand the **Hadoop ecosystem basics** because Spark lives in that world.

What you SHOULD learn?

Learn these at a **conceptual + light hands-on** level:

- **HDFS** → distributed storage
- **YARN** (or K8s) → resource management
- **How Spark works with HDFS/S3**
- **File formats** → Parquet, ORC, CSV

👉 This gives you the “big data foundation” without wasting time on outdated tech.

1) Why Hadoop exists

Before Hadoop, companies stored data in:

- relational databases
- single powerful servers

This worked **until data exploded** because of:

- social media
- mobile apps
- sensors/IoT
- logs
- videos & images
- online transactions

Suddenly data became:

- **too big** to fit on one machine (Volume)
- **coming too fast** (Velocity)
- **in many formats** like JSON, images, logs (Variety)

Traditional systems failed because:

- scaling a single machine is expensive (vertical scaling)
- if the machine crashes → **everything is lost**
- databases couldn't store unstructured data easily
- processing terabytes on one machine takes days

So we needed something new.

👉 **Hadoop was created to:**

- store huge data across **many cheap machines**
- automatically **replicate** data so it's not lost
- **process data in parallel** on multiple machines
- continue working even if some machines fail

It introduced:

- **HDFS** → distributed storage
- **MapReduce** → distributed processing
- **YARN** → resource management

Instead of:

┃ One very big machine

Hadoop uses:

┃ Hundreds of normal machines working together

This is called **horizontal scaling**.

Hadoop exists because traditional single-machine databases could not handle the size, speed, and variety of modern data. Hadoop allows data to be stored across many machines (HDFS) and processed in parallel (MapReduce), while tolerating machine failures and scaling horizontally using clusters of inexpensive hardware. It made it possible to work with terabytes and petabytes of data efficiently.

2) Hadoop ecosystem overview

Know what each does at a **one-line level**:

- **HDFS** – distributed file system for big data storage
- **YARN** – resource manager / job scheduler
- **MapReduce** – older processing engine
- **HBase** – NoSQL database on HDFS
- **Hive** – SQL-like query engine on Hadoop
- **Sqoop** – RDBMS ↔ Hadoop data transfer
- **Flume** – streaming log ingestion
- **Oozie/Airflow** – workflow schedulers

(You don't need to master all — just know what they are)

3) HDFS - Hadoop Distributed File System (very important)

It is a **file system designed to store very large files across many machines**.



Why we need HDFS instead of normal file systems?

Normal disk/file systems:

- store data on **one machine**
- fail if machine dies
- struggle with TB–PB data

HDFS:

- stores huge files by **splitting them into blocks**
 - spreads those blocks across **many machines**
 - keeps **copies** of each block (replication)
 - continues working even if some machines fail
-

Key idea: Blocks

When you store a big file in HDFS:

- it is **not stored as single file**
- it is **broken into fixed-size blocks**
- default block size = **128 MB or 256 MB**

Example:

File size = 500 MB

Block size = 128 MB

It becomes:

- Block 1 → 128 MB
- Block 2 → 128 MB
- Block 3 → 128 MB
- Block 4 → 116 MB

These blocks are:

- stored on **different machines**
 - replicated (usually 3 copies)
-

HDFS has two main actors

1) NameNode – the boss

Stores **metadata only**:

- which file exists
- which blocks belong to file
- which machine has each block

It does **not** store actual data blocks.

Think of it as:

| "File system index / directory manager"

2) DataNode – workers

They:

- store actual blocks of data
- send heartbeat to NameNode
- serve data when requested

So:

- NameNode = **book catalog**
 - DataNode = **shelves holding books**
-

Fault tolerance (very important)

What if one machine dies?

👉 Nothing is lost

Because:

- each block is replicated (default 3 copies)
- NameNode detects dead DataNode
- missing blocks are automatically recreated

This is why Hadoop clusters are **reliable on cheap hardware**

How reading a file works (simple flow)

1. You ask for the file
2. NameNode says:
 - "file has these blocks"
 - "they are on machines X, Y, Z"
3. Client fetches directly from DataNodes
4. Blocks are combined and shown to you

HDFS (Hadoop Distributed File System) stores very large files across many machines by splitting them into blocks and replicating those blocks for fault tolerance. The **NameNode** stores metadata about which blocks belong to which files, while **DataNodes** store the actual data blocks. HDFS provides horizontal scaling, high throughput, and reliability by replicating data so the system continues working even if some machines fail.

HDFS – core concepts:

- blocks
- replication
- NameNode vs DataNode

I'll keep it **clear + practical**, with a short **note block** at the end you can save.

1) HDFS Blocks

Key idea

HDFS does **not** store files as one continuous piece.

Instead, every file is:

- **split into fixed-size pieces** called **blocks**
- stored **across multiple machines**

Default block size:

- earlier: **64 MB**
- now: **128 MB or 256 MB** (configurable)

Example

File size = 500 MB

Block size = 128 MB

It becomes:

- Block 1 → 128 MB
- Block 2 → 128 MB

- Block 3 → 128 MB
- Block 4 → 128 MB
- Block 5 → - remaining - 500 - 512? small remainder 116 MB

So **one logical file** becomes **multiple blocks on multiple nodes**.

Why large block sizes?

- fewer blocks to manage
- faster sequential reads
- optimized for big files (GB-TB)

HDFS is designed for:

✓ **write once, read many times**

✗ not for **frequent random updates**

2) Replication

Every block in HDFS is **automatically copied** multiple times.

This is called **replication**.

Default **replication factor** = 3

Meaning:

👉 each block has **3 copies** on **3 different DataNodes**

✓ Why replication is needed?

- machine may fail
- disk may die
- network may break

Still your data must live 😊

Important rule

Copies are stored on **different nodes** and often **different racks**.

So even if:

- one disk dies → OK
 - one machine dies → OK
 - entire rack dies → still OK
-

3) NameNode vs DataNode

HDFS has two main players.

 NameNode — *the master*

Stores **metadata only**, such as:

- file names
- directories
- permission info
- mapping of files → blocks
- mapping of blocks → DataNodes

It does **NOT** store actual data.

Think of it like:

| Table of contents or index of a big book

If NameNode dies without backup → metadata is lost 😞

That is why we later learn **High Availability**.

 DataNode — *the workers*

They:

- store **actual data blocks**
- read/write blocks on request
- send **heartbeat** to NameNode periodically

If DataNode dies:

👉 nothing is lost

👉 NameNode notices missing heartbeat

👉 blocks get re-replicated elsewhere

Relationship in one line

- **NameNode knows where data is**
- **DataNodes actually hold data**

You NEVER talk to DataNodes directly for metadata.

Clients first ask the **NameNode**, then fetch from **DataNodes**.

In Hadoop, a **cluster** is a group of machines working together, and each machine is usually a **VM (virtual machine)** in cloud platforms. On these VMs, Hadoop services run: the **NameNode service** manages metadata (which blocks belong to which files), and multiple **DataNode services** store the actual data blocks. A large file (e.g., 1 GB = 1024 MB) is split into **blocks** (default 128 MB → 8 blocks) and each block is **replicated** across different DataNodes (default replication factor = 3, configurable) to provide fault tolerance and high availability. So 8 blocks become 24 physical copies across nodes. Services like **AWS EMR or GCP Dataproc** automatically create VMs, install Hadoop/Spark, assign roles (master → NameNode/ResourceManager, workers → DataNode/NodeManager), connect them, and let us **scale up/down worker/DataNodes** easily. In short: VM = single machine, Node = role on that machine, Cluster = many nodes/VMs working together, and replication keeps data safe across DataNodes.

1 GB file → 8 blocks → replicated across 4 DataNodes

Blocks (logical):

B1 B2 B3 B4 B5 B6 B7 B8

4 DataNodes:

DN1 DN2 DN3 DN4

Replication factor = 3 (each block has 3 copies)

(example distribution — Spark/Hadoop place them automatically)

B1 → DN1 DN2 DN3
B2 → DN2 DN3 DN4
B3 → DN1 DN3 DN4
B4 → DN1 DN2 DN4
B5 → DN1 DN2 DN3
B6 → DN2 DN3 DN4
B7 → DN1 DN3 DN4
B8 → DN1 DN2 DN4

So you can **see visually**:

- 8 blocks
- each block stored on **3 different DataNodes**
- total copies = **8 × 3 = 24**

HDFS Reliability 👍

We'll cover three ideas clearly:

1. Fault tolerance
2. Rack awareness
3. High availability (Active/Standby NameNode)

🛡️ 1) Fault tolerance

Fault tolerance = system keeps working even when machines fail.

In HDFS, failures are expected:

- disks crash
- VMs shut down

- network breaks
- entire nodes die

HDFS handles this mainly using:

- **replication**
- **heartbeats**
- **automatic re-replication**

How it works (simple flow)

1. Each DataNode sends **heartbeats** to NameNode regularly.
2. If NameNode doesn't receive heartbeat:
 - it marks DataNode as **dead**
3. Blocks stored on that node now have **fewer copies**
4. NameNode automatically triggers:
 - **re-replication** to other healthy DataNodes
5. Replication factor is restored (e.g., back to 3)

👉 So even if machines die, **data stays available**.

2) Rack awareness

Think of a **rack** as:

- a physical metal frame in a data center
- contains multiple servers/VM hosts
- has a single network switch

Rack awareness means:

Hadoop knows which DataNode belongs to which rack.

? Why does this matter?

Because failures can happen:

- at machine level

- at entire rack level (power/switch failure)

So HDFS places replicas smartly:

- **some copies in same rack** (fast access)
- **some copies in different racks** (safer)

Example placement for replication factor = 3:

- copy 1 → rack A
- copy 2 → rack A
- copy 3 → rack B

This balances:

- network efficiency
- disaster safety

👉 Even if entire rack A goes down, copy in rack B survives.

3) High Availability (Active/Standby NameNode)

Earlier Hadoop versions had a big problem:

| If the single NameNode failed → whole cluster stopped.

So Hadoop introduced **High Availability (HA)**.

HA has two NameNodes:

- **Active NameNode** → currently serving clients
- **Standby NameNode** → backup, in sync, ready to take over

They share:

- metadata edits
- namespace changes
- journal logs

If Active NameNode fails:

- Standby becomes Active automatically
- system continues with **almost no downtime**

This is done using:

- **JournalNodes** (usually 3)
- **Zookeeper failover controller**

Simple analogy to remember

- Replication = photocopying pages
- Rack awareness = keep copies in different rooms
- HA NameNode = second librarian ready if first faints 🤪

HDFS is highly reliable through fault tolerance, rack awareness, and NameNode high availability. Fault tolerance is achieved using replication and heartbeats: if a DataNode fails, the NameNode detects it and automatically re-replicates lost blocks to other nodes. Rack awareness ensures replicas are distributed across different racks so data survives even if an entire rack fails. To avoid single-point failure of the NameNode, Hadoop supports High Availability with an Active NameNode and Standby NameNode; if the active one fails, the standby takes over, keeping the cluster running.

Practical things worth knowing

1) How to check replication factor

Every HDFS file stores replication info.

Command:

```
hdfs dfs -stat %r /path/to/file
```

This prints replication factor of that file.

2) How to change replication factor (per file)

```
hdfs dfs -setrep2 /path/to/file
```

or recursively:

```
hdfs dfs -setrep -R2 /directory/path
```

3) How to check where blocks are stored

```
hdfs fsck /path/to/file -files -blocks -locations
```

This shows:

- blocks
- replicas
- corresponding DataNodes

This directly answers:

| "Which node has which copy?"

4) Know default configs conceptually

Located in:

- `hdfs-site.xml`
- `core-site.xml`

Example properties:

- `dfs.replication` → default replication factor
- `dfs.blocksize` → block size

You don't need to memorize them — just know they exist.



Optional hands-on (if you ever get access)

If you get a Hadoop cluster (local or EMR/Dataproc), try:

- upload a file
- view its blocks
- stop a DataNode
- watch Hadoop re-replicate

But this is **nice-to-have**, not mandatory for now.



Note for your notebook

For replication and reliability, it's useful to know a few practical HDFS commands: `-stat` to view replication, `-setrep` to change replication, and `fsck` to see block locations. Replication factor and block size are configured in `hdfs-site.xml`. Hands-on cluster setup is optional; understanding concepts plus these basic commands is sufficient for intermediate-level big data knowledge.

Writing to HDFS

To write a file to HDFS, the client first contacts the NameNode for block placement. The file is split into blocks, and each block is written using a pipeline: client → DataNode1 → DataNode2 → DataNode3, based on replication factor. Acknowledgements flow back along the pipeline, and only then the write is considered successful. To read a file, the client again contacts the NameNode to get block locations, then reads blocks directly from the nearest DataNodes. The NameNode only manages metadata and never stores file data; DataNodes handle the actual read/write of data blocks.

When reading data from HDFS, the client asks the NameNode for the locations of each block. The NameNode does not return all DataNodes; it returns only the specific nodes that actually store replicas of that block (e.g., for B2 → DN2, DN3, DN4). The client then selects the nearest node among those replicas and reads the block. Therefore, the client never contacts a DataNode that doesn't contain the block, and the file is reconstructed block-by-block from the correct DataNodes.

HDFS does not require one DataNode to store a full file. A file is split into blocks, and each block has its own replicas across different DataNodes. When reading, the client asks the NameNode for the list of DataNodes for every block, then reads

each block from one of its replica nodes (possibly different nodes for different blocks), and finally reassembles the blocks in order to reconstruct the full file.

In day-to-day work, I don't need to manually control how HDFS reads or writes data — I simply use commands or Spark APIs to read/write files. However, understanding how HDFS internally writes (pipeline replication) and reads (per-block from nearest replica) is important for debugging failures, performance tuning, interviews, and truly understanding why Hadoop/Spark behave the way they do.

Basic HDFS Commands You Should Know



1) Upload (write) a file to HDFS

Upload from local machine → HDFS:

```
hdfs dfs -put mydata.csv /user/sunny/
```

or overwrite if already exists:

```
hdfs dfs -copyFromLocal -f mydata.csv /user/sunny/
```



2) Download (read) a file from HDFS

HDFS → local machine:

```
hdfs dfs-get/user/sunny/mydata.csv .
```

(downloads into current folder)

3) List files in HDFS directory

```
hdfs dfs -ls /user/sunny/
```

Recursive list:

```
hdfs dfs -ls -R /user/sunny/
```

4) View file content

Print like `cat` :

```
hdfs dfs -cat /user/sunny/mydata.csv
```

View first few lines:

```
hdfs dfs -head /user/sunny/mydata.csv
```

View last few lines:

```
hdfs dfs -tail /user/sunny/mydata.csv
```

5) Delete file or folder

Delete file:

```
hdfs dfs -rm /user/sunny/mydata.csv
```

Delete folder recursively:

```
hdfs dfs -rm -r /user/sunny/myfolder
```

Force delete (skip trash):

```
hdfs dfs -rm -r -skipTrash /user/sunny/myfolder
```



6) Create directories

```
hdfs dfs -mkdir /user/sunny/input
```

Create parents automatically:

```
hdfs dfs -mkdir -p /user/sunny/projects/bigdata
```



7) Check replication & block info (important)

Check replication factor of file

```
hdfs dfs -stat %r /user/sunny/mydata.csv
```

See block locations

```
hdfs fsck /user/sunny/mydata.csv -files -blocks -locations
```

This is how you see:

- which blocks exist

- which DataNodes store them

8) Change replication factor

Per file:

```
hdfs dfs -setrep 2 /user/sunny/mydata.csv
```

Recursively for folder:

```
hdfs dfs -setrep -R 3 /user/sunny/
```



Concise notes for your notebook

Common HDFS commands: -put and -get to upload/download files, -ls to list, -cat/head/tail to view content, -mkdir to create directories, and -rm to delete. To work with replication, use -stat %r to view replication factor and -setrep to change it. Use fsck -files -blocks -locations to see block and DataNode locations. These are the essential practical commands for daily HDFS usage.

4) Cluster concepts & YARN basics (how jobs run)



1) What is YARN?

YARN = **Yet Another Resource Negotiator**

It is the **resource manager** in Hadoop.

Simple meaning:

YARN decides who gets how much CPU & RAM and where jobs run in the cluster.

Think of YARN as:

- cluster **traffic controller**
- job **scheduler**
- resource **allocator**

It doesn't process data itself — it **manages** the machines that do.

2) Why do we need YARN?

In a cluster:

- multiple users
- multiple jobs (Spark, MapReduce, Hive, etc.)
- shared machines

Without YARN:

- ✗ two big jobs may crash each other
- ✗ one job may eat entire cluster
- ✗ no centralized scheduling
- ✗ difficult tracking of job status

With YARN:

- ✓ fair sharing
 - ✓ resource limits
 - ✓ parallel jobs
 - ✓ monitoring & retry
 - ✓ queues and priorities
-

3) Main Components of YARN

There are **three key players**.

A) ResourceManager (RM) — the boss

Runs **on master node**

Responsible for:

- tracking cluster resources
- deciding **which job runs where**
- scheduling containers

Think:

| ResourceManager = CEO of the cluster

B) NodeManager (NM) — local manager

Runs **on every worker/DataNode**

Responsible for:

- actually launching tasks (containers)
- reporting resource usage to RM
- monitoring containers
- sending heartbeats

Think:

| NodeManager = team leader of each machine

C) Containers — where work runs

A **container is a slice of machine resources**

Contains:

- some RAM

- some CPUs
- environment to run a task

Think:

| Container = “room” allocated in a building to run a job

Spark executors / map tasks / reduce tasks **run inside containers**.

4) How a Spark job runs on YARN (simple flow)

Let's assume you submit a Spark job.

Step-by-step:

- 1 You submit the job (spark-submit, Hive query etc.)
- 2 ResourceManager receives request
- 3 RM allocates a **container** for **ApplicationMaster**
- 4 ApplicationMaster requests more containers for executors/tasks
- 5 NodeManagers launch those containers on different nodes
- 6 Tasks run in parallel
- 7 Results collected → job finishes
- 8 YARN releases resources

Key takeaway:

👉 **YARN does scheduling; Spark/Hadoop do computation**

Small analogy

- ResourceManager = company HR assigning people to rooms
- NodeManager = office admin on each floor
- Container = room with chairs and tables

- Spark executor = team working inside room



Concise note for your notebook

YARN (Yet Another Resource Negotiator) is Hadoop's cluster resource manager and job scheduler. It decides how CPU and memory are shared across jobs and nodes. The ResourceManager (on master) allocates resources; NodeManagers (on each worker) launch and monitor containers; containers are resource bundles where tasks actually run. When a Spark or Hadoop job is submitted, YARN allocates containers across the cluster and manages their lifecycle, enabling multiple users and jobs to safely share cluster resources.

5) MapReduce — only conceptually

Assume a text file in HDFS (1 GB or any size):

```
bigdatais big  
datais powerful
```



Map Phase

Input is split across blocks → each **Mapper** processes some lines.

Mapper reads text and outputs (word, 1) pairs

For line:

```
bigdatais big
```

Mapper emits:


```
(big,1)
(data,1)
(is,1)
(big,1)
```

For line:

```
datais powerful
```

Mapper emits:

```
(data,1)
(is,1)
(powerful,1)
```

After all Mappers finish, we conceptually have:

```
(big,1) (data,1) (is,1) (big,1) (data,1) (is,1) (powerful,1)
```

Shuffle & Sort Phase

Hadoop automatically:

- groups same words together
- sends them to the same reducer

Grouped output becomes:

```
big →1,1
data →1,1
is →1,1
```

powerful →1



Reduce Phase

Reducer now **adds the counts**.

Examples:

big →1 +1 =2
data →1 +1 =2
is →1 +1 =2
powerful →1

Reducer outputs:

(big,2)
(data,2)
(is,2)
(powerful,1)



Final output written to HDFS

big2
data2
is2
powerful1



Key takeaways

- Mapper → breaks text into words → emits (word, 1)
 - Shuffle → groups same words together
 - Reducer → adds up counts per word
 - Output → word frequencies in the file
-



Short note to save

In the classic Word Count MapReduce program, Mappers read text input and emit (word, 1) for every word they see. During shuffle and sort, Hadoop groups values of the same word together. Reducers then sum these values to produce total counts such as (big, 2) and (data, 2). Input comes from HDFS and final counts are written back to HDFS. Map = emit word occurrences; Reduce = aggregate counts per word.

6) File formats (very practical)

Know pros/cons of:

- CSV
- JSON
- Parquet (priority)
- ORC
- Avro

Key ideas:

- columnar vs row-based
- compression
- schema evolution

7) Data ingestion basics

How data arrives into Hadoop:

- batch ingestion
- streaming ingestion
- tools: Kafka, Flume, Sqoop (conceptual only)

Apache Spark

1) What is Apache Spark?

Simple definition

Apache Spark is a fast, distributed computing engine for processing big data, supporting SQL, streaming, machine learning, and batch processing — usually in-memory.

More practically:

- Hadoop MapReduce = batch, disk-based, slow
- **Spark = in-memory, fast, flexible**

Spark supports:

- Spark SQL ← you already used this 
- DataFrame API
- RDD (low-level)
- Spark Streaming
- MLlib (machine learning)
- GraphX

Spark runs on top of:

- YARN
- Kubernetes
- Mesos

- Standalone cluster
- Local mode (even on laptop)

It reads data from:

- HDFS
 - Hive tables
 - Kafka
 - S3 / GCS
 - RDBMS
 - Local files
-

2) Spark Architecture — very important

Spark has **three key parts**:

Driver program

The “boss” program that:

- holds your SparkSession
- plans the job
- talks to Cluster Manager
- sends tasks to executors
- collects results

Your **Java / Scala / Python program = Driver.**

Cluster Manager

Allocates resources.

Can be:

- YARN
- Kubernetes

- Standalone Spark
- Mesos

It decides:

“Which machine gets how many executors and memory?”

Executors

Actual workers.

They:

- run the tasks
- store data in memory
- return results to driver

So architecture looks like:

Driver → Cluster Manager → Executors

Understand RDD's [Resilient Distributed Dataset]

https://github.com/backstreetprogrammer/11_JavaSpark#16-spark-rdds

First: What they actually are

✓ RDD

```
RDD<T>
```

Example:

```
RDD<Integer>  
RDD<String>  
RDD<Person>
```

Generic distributed collection.

✓ Pair RDD

```
RDD<Tuple2<K,V>>
```

Example:

```
RDD<(word,1)>  
RDD<(city, sales)>  
RDD<(userId, listOfClicks)>
```

Spark calls these **Pair RDDs** because they store **(key, value)** pairs.

They are still RDDs — just a *special case of RDD*.

BIG IDEA (important)

PairRDD is powerful not because of partitions
but because **Spark can group/aggregate/join using the key**

Because now Spark knows:

- what is **key**
- what is **value**

So Spark can:

- shuffle by key
 - combine values by key
 - reduce each key in parallel
 - join datasets by key
 - group by key
-

Your assumption (small correction)

You said:

if i want to calculate on individual partition then rdd

if i want to perform action on top of all partitions then pair rdd

Not quite 😊

Both RDD and PairRDD:

- exist across all partitions
- operate across distributed cluster
- support per-partition APIs (`mapPartitions` , `foreachPartition` , etc.)

PairRDD is NOT about:

- per partition vs all partitions 
- local vs global 

PairRDD **is about having keys** 

 **Correct understanding should be:**

Question	Correct answer
Need plain distributed list?	RDD
Need key-based operations?	PairRDD
Need group/join/aggregate by something?	PairRDD
Need typed objects?	Dataset
Need SQL?	Spark SQL

What Pair RDD gives you (that normal RDD cannot)

These are **only available on PairRDD**:

- `reduceByKey`
- `groupByKey`
- `mapValues`
- `flatMapValues`
- `combineByKey`
- `countByKey`
- `join`
- `leftOuterJoin`
- `rightOuterJoin`
- `cogroup`

Why?

Because they all require **keys**.

Simple example

RDD (plain list)

```
[10, 20, 30, 40]
```

You can:

- map
- filter
- reduce

PairRDD

```
("apple",1)  
("banana",1)  
("apple",1)
```

You can now:

- ✓ count apples
- ✓ group fruits
- ✓ join with price table
- ✓ reduce per fruit

Example:

```
reduceByKey(sum)
```

Result:

```
("apple",2)  
("banana",1)
```

Key performance insight

PairRDD enables **map-side combine** (super important)

`reduceByKey` :

- combines values **inside partition first**
- then shuffles minimal data
- highly scalable

`groupByKey` :

- shuffles everything (expensive)
- no pre-aggregation

So PairRDD isn't just "more powerful" — it's also **more efficient** for keyed aggregations.

When to use what

Use RDD

- plain unstructured data
- custom algorithms
- non key-value data
- graph-style iteration
- map/filter only

Use PairRDD

- **word count**
- **log analytics**
- **transaction totals per user**
- **join two datasets**
- **group by key**

- **count by key**
- **sum by key**

Example mapping to real life

Problem	API
count words	PairRDD
sales per city	PairRDD
top N salaries	DataFrame / SQL
machine learning algorithm internals	RDD
ETL pipeline	DataFrame
stream join	PairRDD/DataFrame

Part II — Spark DataFrames & Datasets (Java)

1. Introduction to DataFrames & Datasets

- What are DataFrames?
- What are Datasets?
- Differences between:
 - RDD vs DataFrame vs Dataset
- When to use which API
- Tungsten & Catalyst overview (high level)

2. Creating DataFrames

- From JSON / CSV / Parquet
- From Collections
- From RDD

- From external databases (JDBC)
 - Schema introspection (`printSchema()`)
-

3. DataFrame Schema & Types

- StructType / StructField
- Infer schema vs explicit schema
- Nullable vs non-nullable fields
- Encoders

♦ 1. First — NULL vs empty string

Important difference:

value in file Spark sees
, , empty string ""
, , and cast fail null
NULL text "NULL" string
missing column null

Check which is which:

```
emp.select(  
  col("salary"),  
  col("salary").isNull().alias("is_null"),  
  col("salary").equalTo("").alias("is_empty")  
).show();
```

So:

isNull → true only for real NULL

"" → NOT null

- ♦ 2. dropna — remove rows with null values

Remove rows where any column is null

```
emp.na().drop().show();
```

Remove rows only if specific columns null

```
emp.na().drop(new String[]{"salary"}).show();
```

Require minimum number of non-null columns

```
emp.na().drop(3).show(); // keep rows with >= 3 non-null values
```

- ♦ 3. fillna — replacing NULLS with defaults

Replace all numeric nulls

```
emp.na().fill(0).show();
```

Replace specific column nulls

```
emp.na().fill(0, new String[]{"salary"}).show();
```

Replace string nulls

```
emp.na().fill("UNKNOWN", new String[]{"department"}).show();
```

Replace multiple columns with different defaults

```
emp.na().fill(Map.of(
    "department", "UNKNOWN",
    "salary", 0
));
```

- ♦ 4. replace — value to value replacement (not just null)

Example convert "HR" to "Human Resources":

```
emp.na().replace("department",
    Map.of("HR", "Human Resources")
).show();
```

Important:

replace works on actual values

fillna works on null values

- ♦ 5. Detect & clean blank strings vs nulls

Blank strings first:

```
emp.filter(col("department").equalTo("")).show();
```

Convert blank strings → null:

```
Dataset<Row> cleaned = emp.withColumn(  
    "department",  
    when(trim(col("department")).equalTo(""), null)  
    .otherwise(col("department"))  
);
```

Now dropna works cleanly.

- ♦ 6. _corrupt_record — why it appears

Occurs when:

malformed JSON

bad CSV depending on options

extra columns

missing quotes

Example JSON:

```
[  
{"name":"Amit","age":23}  
BAD_JSON  
{"name":"Sunny","age":27}  
]
```

Read with permissive mode (default):

```
Dataset<Row> df = spark.read()  
    .option("multiLine", true)  
    .json("people.json");
```

Result includes:

_corrupt_record	name	age
-----------------	------	-----

The bad row goes into _corrupt_record.

Remove corrupted rows:

```
df.filter(col("_corrupt_record").isNull()).show();
```

- ♦ 7. CSV read modes — important for interviews

Default — PERMISSIVE (most forgiving)

```
.option("mode", "PERMISSIVE")
```

Behavior:

malformed rows → _corrupt_record

wrong types → converted to null

no job failure

DROPMALFORMED — just drop bad rows

```
.option("mode", "DROPMALFORMED")
```

Bad rows are simply skipped.

FAILFAST — crash fast

```
.option("mode", "FAILFAST")
```

Throws exception ONLY IF:

malformed record structure

NOT just null or empty

Remember from earlier:

FAILFAST does NOT fail on null-in-non-nullable column

- ♦ 8. Real-world null-handling pattern

Production pipelines typically do:

```
Dataset<Row> cleaned = emp
  // normalize blanks → null
  .withColumn("department",
    when(trim(col("department")).equalTo(""), null)
    .otherwise(col("department")))
  // reject bad rows
  .filter(col("salary").isNotNull())
  // or default
  .na().fill(0, new String[]{"salary"});
```

Then:

- ✓ no null salary
- ✓ bad records removed or treated
- ✓ clean stable schema

4. DataFrame Basic Operations

- `select`
- `withColumn`
- `filter` / `where`
- `groupBy`
- `orderBy` / `sort`
- `agg`

5. Column Expressions

- `col()` and `column()`
 - Expression strings `"salary * 2"`
 - Built-in functions (`functions.*`)
 - `lit`
 - `upper`
 - `substring`
 - `concat`
 - `when` / `otherwise`
-

6. Handling Missing & Bad Data

- `dropna`
 - `fillna`
 - `replace`
 - detect null vs blank
 - corrupt record handling (`_corrupt_record`)
-

7. Joins with DataFrames

- inner, left, right, full
 - semi / anti joins
 - broadcast join
 - join condition vs natural column name join
 - duplicate column resolution
-

8. Grouping & Aggregations

- `groupBy`
 - `rollup`
 - `cube`
 - window functions:
 - rank
 - row_number
 - dense_rank
 - moving averages
-

9. Saving & Loading Data

- JSON

- CSV
 - Parquet
 - ORC
 - Partitioning
 - Bucketing
-

10. Performance & Optimization

- Catalyst optimizer intuition
 - Tungsten execution engine
 - Predicate pushdown
 - Explain plans
 - Caching / persisting DataFrames
 - Repartition vs Coalesce (DataFrame version)
-

11. Dataset API (Strongly-Typed)

- Java POJOs → Dataset
 - Encoders
 - Typed `map`, `filter`, `flatMap`
 - When to prefer Dataset vs DataFrame
-

12. Structured Streaming (Intro)

- Static DataFrame vs Streaming DataFrame
- Source examples:
 - file
 - socket
- Output sinks

- console
 - file
 - watermark basics
-

Mini-Projects for DataFrames

- Movie analytics (ratings CSV)
 - Sales aggregation dashboard
 - Log analysis
 - Taxi ride dataset analysis
 - Null / bad data cleansing project
-

Part III — Spark SQL

1. Introduction to Spark SQL

- What is Spark SQL?
- Relation with DataFrames
- Temporary views vs Global views
- Catalogs & databases

★ Topic 1 — Spark SQL Fundamentals (START HERE)

- ♦ What is Spark SQL?

Spark SQL is:

A SQL execution engine built on top of Spark's DataFrame engine.

Important truth:

Spark SQL and DataFrame API are the SAME engine.

Only the syntax is different.

🧠 Think of it this way

DataFrame API Spark SQL

`df.filter(col("salary") > 60000)` `WHERE salary > 60000`

`df.groupBy("dept").avg("salary")` `GROUP BY dept`

`df.join(df2, "id")` `JOIN`

window functions `OVER(PARTITION BY...)`

Same execution plan ✓

- ♦ First Spark SQL program (very simple)

Step 1: Read DataFrame

```
Dataset<Row> emp = spark.read()  
    .option("header", true)  
    .option("inferSchema", true)  
    .csv("src/main/resources/employees.csv");
```

Step 2: Register Temp View

```
emp.createOrReplaceTempView("employees");
```

This makes employees visible to SQL.

Step 3: Run SQL query

```
spark.sql("""  
    SELECT name, salary  
    FROM employees  
    WHERE salary > 60000  
    """).show();
```

That's it. You just ran Spark SQL 🎉

Very Important: Temp View Scope

Exists only in this SparkSession

Lost when app stops

Not visible across applications

- ♦ SQL vs DF — same plan proof

```
emp.filter(col("salary").gt(60000)).explain(true);
```

```
spark.sql("SELECT * FROM employees WHERE salary > 60000")  
  .explain(true);
```

You'll see same logical & physical plan.

2. Registering DataFrames as Tables

- `createOrReplaceTempView`
- `createGlobalTempView`
- Using `spark.sql("...")`

- ♦ Spark SQL – Topic 2

Temporary Views vs Global Temporary Views

1 Why views exist in Spark SQL

Spark SQL works on DataFrames internally.

Views are just names (aliases) that allow SQL to reference DataFrames.

Views do NOT store data

They only point to a DataFrame

2 Temporary View (Most Common)

✓ How to create

```
emp.createOrReplaceTempView("employees");
```

✓ How to use

```
spark.sql("""
  SELECT department, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department
""").show();
```

🔍 Scope & lifetime

Property Temporary View

Scope SparkSession only

Lifetime Until SparkSession stops

Visible in another session? ❌ No

Stored on disk? ❌ No

🧠 Mental model

Temporary View ≈ local variable inside a method

3 Global Temporary View

✓ How to create

```
emp.createOrReplaceGlobalTempView("employees");
```

! How to query (important!)

Global views live under global_temp database

```
spark.sql("""
  SELECT *
  FROM global_temp.employees
""").show();
```

🔍 Scope & lifetime

Property Global Temp View

Scope Spark Application

Lifetime Until Spark application stops
Visible in another SparkSession?  Yes
Stored on disk?  No
 Mental model

Global Temp View $\approx$ static variable shared across objects

4 Temp vs Global — Side-by-side

Feature	Temp View	Global Temp View
Created by	<code>createOrReplaceTempView</code>	<code>createOrReplaceGlobalTempView</code>
Accessed as	<code>employees</code>	<code>global_temp.employees</code>
Scope	SparkSession	Spark Application
Multiple sessions?		
Needs DB prefix		

5 Example with 2 SparkSessions (important)

```
SparkSession spark1 = SparkSession.builder().getOrCreate();  
SparkSession spark2 = spark1.newSession();
```

```
emp.createOrReplaceTempView("employees");
```

```
spark1.sql("SELECT * FROM employees").show(); //   
spark2.sql("SELECT * FROM employees").show(); // 
```

Now global:

```
emp.createOrReplaceGlobalTempView("employees");
```

```
spark1.sql("SELECT * FROM global_temp.employees").show(); //   
spark2.sql("SELECT * FROM global_temp.employees").show(); // 
```

6 When should YOU use which?

 Use Temp View (90% cases)

single Spark job

notebooks

local development

batch jobs

✔ Use Global Temp View when

multiple SparkSessions

shared SQL logic

Spark Thrift Server

complex pipelines

7 Interview-ready one-liner

Temp view is session-scoped; global temp view is application-scoped and lives under global_temp database.

3. SQL Querying Fundamentals

- select, where
- group by, having
- order by, limit
- distinct

4. SQL Joins in Spark

- cross join
- inner / outer joins
- handling duplicate column names

5. SQL Functions Library

- string functions
 - date functions
 - numeric functions
 - JSON functions
 - null functions
-

6. User-Defined Functions (UDFs)

- Register Java UDF
 - Use inside SQL
 - Performance drawbacks
 - When not to use UDFs
 - Pandas UDF intro (conceptual)
-

7. Views, Databases & Tables

- Managed vs External tables
 - Metastore concept
 - Warehouse dir
 - Persisting tables
-

8. Spark SQL on Hive (Optional but Useful)

- Enable Hive support
- Create database & tables
- Query Hive tables from Spark
- Insert into Hive from Spark SQL

Apache Hive — Intermediate Learning Path (with Spark)

1 What is Hive?

Apache Hive is a data warehouse layer built on top of distributed storage (HDFS / cloud storage).

Think of Hive as:

SQL on files, not on row-based databases.

Key idea

Data is stored as files (CSV / Parquet / ORC / JSON)

Hive provides:

Schema

SQL

Metadata

Hive does not move data

Hive does not store data itself

2 Hive vs RDBMS (Very Important)

Feature	RDBMS	Hive
---------	-------	------

Storage	Local disk	HDFS / S3 / ADLS
---------	------------	------------------

Data	Row-based	File-based
------	-----------	------------

Schema	Enforced	Schema-on-read
--------	----------	----------------

Updates	Fast	Limited / slow
---------	------	----------------

Use case	OLTP	OLAP / Analytics
----------	------	------------------

 Hive is NOT for transactions

👉 Hive IS for analytics & reporting

3 What does Hive actually store?

Hive stores metadata only:

database name

table name

column names & types

file location

partition info

📌 Stored in Metastore (usually a relational DB)

4 How your emp.csv fits into Hive

Right now:

emp.csv → Spark DataFrame → Temp View → Spark SQL

With Hive:

emp.csv → Hive Table (metadata)

↳ Spark SQL reads Hive table

Important:

Hive table = schema + pointer to file location

5 Spark SQL on Hive (Why Spark + Hive?)

Spark SQL can:

read Hive tables

write Hive tables

replace MapReduce-based Hive execution

So in real projects:

Spark executes, Hive catalogs

6 Enable Hive support in Spark

- ♦ Change SparkSession

```
SparkSession spark = SparkSession.builder()  
    .appName("SparkHiveIntro")  
    .master("local[*]")  
    .enableHiveSupport()  
    .getOrCreate();
```

This does 3 things:

enables Hive SQL

loads Hive metastore

allows CREATE DATABASE, CREATE TABLE

7 Create Hive database

```
spark.sql("CREATE DATABASE IF NOT EXISTS company_db");  
spark.sql("USE company_db");
```

Check:

```
spark.sql("SHOW DATABASES").show();
```

8 Create Hive table from emp.csv (EXTERNAL)

emp.csv (same as before)

id,name,department,salary,city

1,Amit,IT,50000,hyd

2,Sunny,IT,65000,kmm

3,Priya,HR,55000,vij

4,Rahul,Finance,70000,spt

5,Neha,HR,60000,kmm

Create table

```
spark.sql(  
  "CREATE EXTERNAL TABLE IF NOT EXISTS employees (" +  
  "id INT, " +  
  "name STRING, " +  
  "department STRING, " +  
  "salary INT, " +  
  "city STRING" +  
  ") " +  
  "ROW FORMAT DELIMITED " +  
  "FIELDS TERMINATED BY ',' " +  
  "STORED AS TEXTFILE " +  
  "LOCATION 'file:/absolute/path/to/emp_folder'"  
);
```

 Important:

LOCATION points to folder, not file

Spark/Hive will read all files in that folder

9 Query Hive table from Spark SQL

```
spark.sql("SELECT * FROM employees").show();
```

```
spark.sql(  
  "SELECT department, AVG(salary) " +  
  "FROM employees " +  
  "GROUP BY department"  
).show();
```

At this point:

SQL is Hive SQL

Execution is Spark

Metadata is Hive

 Very Important Mental Model

Spark SQL engine

↑

Hive Metastore (schema)

↑

CSV / Parquet files

10 Insert into Hive table using Spark SQL

Create MANAGED table

```
spark.sql(  
  "CREATE TABLE IF NOT EXISTS employees_managed (" +  
  "id INT, " +  
  "name STRING, " +  
  "department STRING, " +  
  "salary INT, " +  
  "city STRING" +  
  ") STORED AS PARQUET"  
);
```

Insert data


```
spark.sql(
  "INSERT INTO employees_managed " +
  "SELECT * FROM employees"
);
```

Now:

Data is copied into Hive warehouse

Hive manages lifecycle

1 1 External vs Managed tables (CRITICAL)

Feature	External	Managed
Data owned by Hive	✗	✓
Drop table deletes data	✗	✓
Use case	Raw / shared data	Curated data

1 2 Verify storage location

```
spark.sql("DESCRIBE FORMATTED employees").show(50, false);
```

Look for:

Location:

1 3 Real-world usage pattern (IMPORTANT)

Bronze / Silver / Gold

External tables → raw data

Managed tables → processed data

Spark SQL → transformations

This is industry standard.

Managed vs External Tables (Simple Explanation)

 First, one golden rule

Hive tables do NOT store data

Hive tables only describe data

Actual data = files

Hive table = metadata (schema + location)

Real-life analogy (BEST way to understand)

Imagine Hive = Library system

Books = data files

Catalog entry = Hive table

Managed Table (Hive owns the books)

Think of:

Books stored inside the library

What this means:

Hive decides where data lives

Hive controls the data

Hive cleans up data

In practice:

```
df.write().saveAsTable("employees");
```

Hive does:

```
C:/spark-warehouse/company_db.db/employees/
```

└─ part-0000.parquet
└─ part-0001.parquet

Behavior:

Action Result

Create table Hive creates folder + files

Drop table Hive deletes files

Move files ❌ Don't touch

When to use:

- ✓ Processed data
- ✓ Reporting tables
- ✓ Final analytics data

📁 External Table (Hive only knows the address)

Think of:

Books stored at your home, not library

What this means:

Hive does NOT own the data

Hive only points to data

Hive never deletes data

In practice:

```
df.write()  
  .option("path", "C:/data/employees")  
  .saveAsTable("employees_ext");
```

Hive does:

C:/data/employees/

|— part-0000.parquet

Hive just remembers:

employees_ext → C:/data/employees

Behavior:

Action Result

Create table Hive registers metadata

Drop table Files remain

Delete files Hive breaks

When to use:

- ✓ Raw data
- ✓ Shared datasets
- ✓ Data managed by other systems

🔥 Side-by-side comparison (VERY IMPORTANT)

Feature Managed External

Who owns data Hive You / other system

Where stored Hive warehouse Custom location

Drop table Deletes data Keeps data

Data safety Less More

Typical layer Gold / Silver Bronze

🔄 Spark Mental Mapping

Managed

```
df.write().saveAsTable("table_name");
```

External

```
df.write()  
  .option("path", "/external/location")  
  .saveAsTable("table_name");
```

That one option changes everything.

⚠ Common beginner mistake

- ✗ Storing raw data as managed table
- ✗ Accidentally dropping table → data gone

Always:

External = raw

Managed = processed

🎯 Interview-ready one-liner

Managed tables are owned by Hive and data is deleted when table is dropped; External tables only store metadata and data is preserved.

```
package com.rayala.hive;

import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.Session;

public class HiveTableExamples {

    public static void main(String[] args) {

        SparkSession spark = SparkSession.builder()
            .appName("HiveTableExamples")
            .master("local[*]")
            .enableHiveSupport()
            .config("spark.sql.warehouse.dir", "file:///C:/spark-warehouse")
            .getOrCreate();
```

```

// Read CSV once
Dataset<Row> empDF = spark.read()
    .option("header", true)
    .option("inferSchema", true)
    .csv("src/main/resources/emp.csv");

spark.sql("CREATE DATABASE IF NOT EXISTS company_db");
spark.sql("USE company_db");

createManagedTable(spark, empDF);
createExternalTable(spark, empDF);

spark.stop();
}

/**
 * Creates a HIVE MANAGED table.
 *
 * - Hive owns the data
 * - Data is stored inside spark-warehouse
 * - Dropping table deletes data
 */
private static void createManagedTable(SparkSession spark, Dataset<Row
> empDF) {

    // Always drop first to avoid location-exists error
    spark.sql("DROP TABLE IF EXISTS employees_managed");

    empDF.write()
        .mode("overwrite")
        .saveAsTable("employees_managed");

    System.out.println("Managed table data:");
    spark.sql("SELECT * FROM employees_managed").show();
}

```

```

/**
 * Creates a HIVE EXTERNAL table.
 *
 * - Hive does NOT own the data
 * - Data stored at custom location
 * - Dropping table keeps data
 */
private static void createExternalTable(SparkSession spark, Dataset<Row>
empDF) {

    spark.sql("DROP TABLE IF EXISTS employees_external");

    empDF.write()
        .mode("overwrite")
        .option("path", "file:///C:/external-data/employees")
        .saveAsTable("employees_external");

    System.out.println("External table data:");
    spark.sql("SELECT * FROM employees_external").show();
}
}

```

◆ What is a Hive Partition?

Simple definition:

Partition = physical folder split based on a column

Instead of one big table folder, Hive creates **sub-folders**.

✗ Without Partition (slow)

```
employees/  
├── part-0000.parquet  
├── part-0001.parquet
```

Query:

```
SELECT * FROM employees WHERE department='IT';
```

👉 Hive must scan **ALL files**

✅ With Partition (fast)

Partition by `department` :

```
employees_partitioned/  
├── department=IT/  
│   ├── part-0000.parquet  
├── department=HR/  
│   ├── part-0000.parquet  
├── department=Finance/  
│   ├── part-0000.parquet
```

Query:

```
SELECT * FROM employees_partitioned WHERE department='IT';
```

👉 Hive reads **ONLY** `department=IT` folder

This is called **partition pruning** 🚀

💠 When should you partition?

Partition columns should be:

✓ Low cardinality

- ✓ Frequently used in filters
- ✓ Business dimensions

Good partition columns

- date
- country
- department
- region

✗ Bad partition columns

- employee_id
 - name
 - UUID
-

◆ Two ways to create partitioned tables

1 SQL style (Hive way)

```
CREATE TABLE employees_part (  
  idINT,  
  name STRING,  
  salaryINT  
)  
PARTITIONEDBY (department STRING);
```

2 Spark style (what you'll use)

```
df.write()  
  .partitionBy("department")  
  .saveAsTable("employees_part");
```



Hands-on: Partitioned Hive Table in Java



emp.csv

```
id,name,department,salary
1,Amit,IT,50000
2,Sunny,IT,65000
3,Priya,HR,55000
4,Rahul,Finance,70000
5,Neha,HR,60000
```



Java Example (Partitioned Managed Table)

```
package com.rayala.hive;

import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.Session;

public class HivePartitionExample {

    publicstaticvoidmain(String[] args) {

        SparkSessionspark= SparkSession.builder()
            .appName("HivePartitionExample")
            .master("local[*]")
            .enableHiveSupport()
            .config("spark.sql.warehouse.dir","file:///C:/spark-warehouse")
            .config("javax.jdo.option.ConnectionURL",
                "jdbc:derby;;databaseName=C:/spark-metastore/metastore_db;create=true")
            .getOrCreate();

        Dataset<Row> empDF = spark.read()
```

```
.option("header",true)
.option("inferSchema",true)
.csv("src/main/resources/emp.csv");

spark.sql("CREATE DATABASE IF NOT EXISTS company_db");
spark.sql("USE company_db");

spark.sql("DROP TABLE IF EXISTS employees_part");

empDF.write()
  .mode("overwrite")
  .partitionBy("department")
  .saveAsTable("employees_part");

spark.sql("SELECT * FROM employees_part").show();

spark.stop();
}
}
```

What gets created on disk

```
C:/spark-warehouse/company_db.db/employees_part/
|— department=IT/
|— department=HR/
|— department=Finance/
```

Partition Pruning (IMPORTANT)

Run:

```
spark.sql(
  "SELECT * FROM employees_part WHERE department = 'IT'"
).explain(true);
```

You'll see:

```
PartitionFilters: [department = IT]
```

That means Spark is **not scanning everything**.

Common mistakes (avoid these)

- ✗ Too many partitions
- ✗ Partitioning on high-cardinality columns
- ✗ Forgetting to filter on partition column
- ✗ Using `repartition()` thinking it's partitioning (not same!)

Partition vs Repartition (important distinction)

Feature	Hive Partition	Spark Repartition
Purpose	Data skipping	Parallelism
Physical folders	Yes	No
Metadata aware	Yes	No
Query optimization	Yes	No

Interview one-liner

Partitioning improves query performance by physically organizing data so that only relevant partitions are scanned.

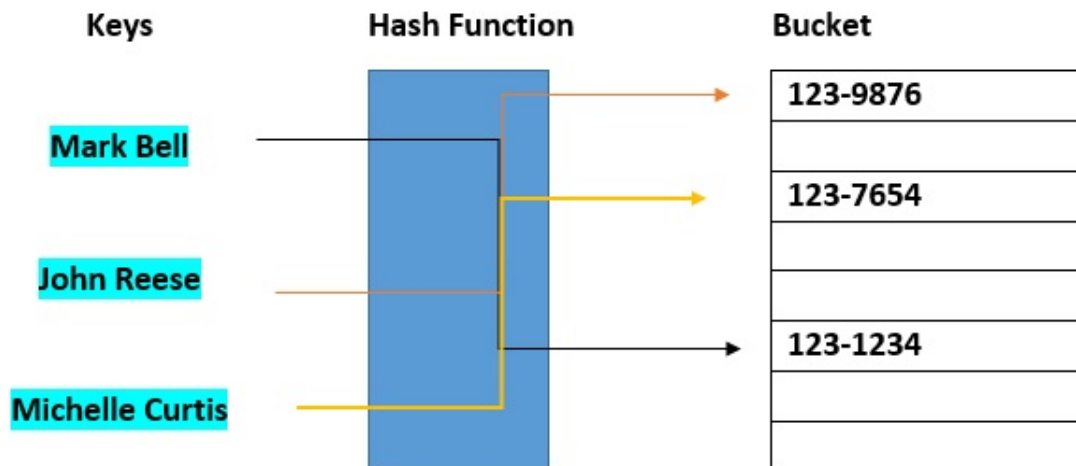
What is Bucketing? (Simple words)

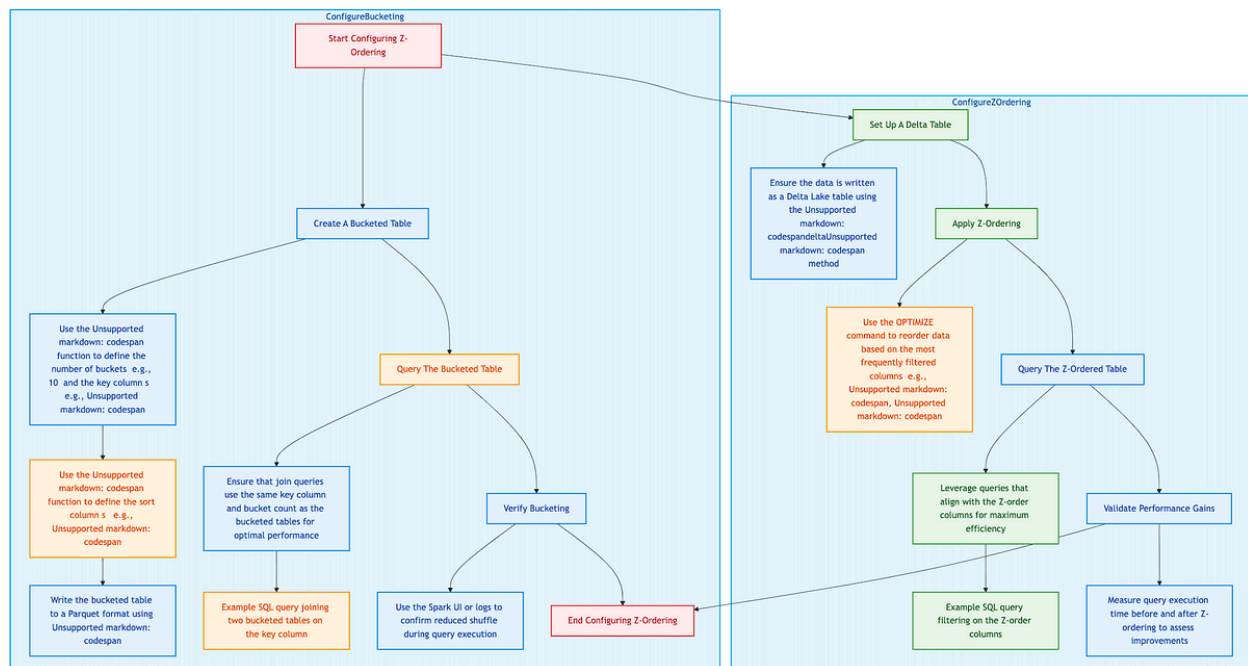
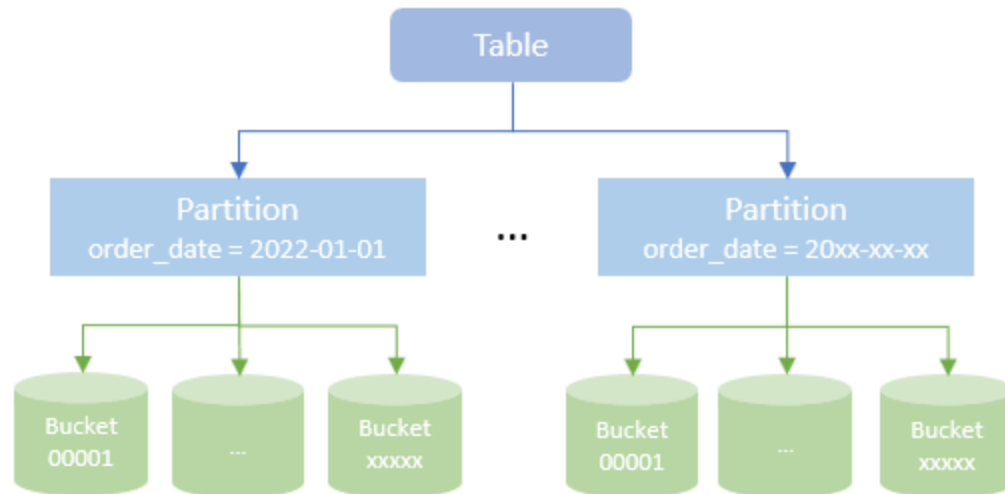
Bucketing = divide data into fixed number of files based on hash of a column

Partitioning vs Bucketing (1-line truth)

Feature	Partitioning	Bucketing
Folder based	✓ Yes	✗ No
Fixed number of files	✗ No	✓ Yes
Best for	filtering	joins
Column type	low-cardinality	high-cardinality

Visual intuition





$\text{bucket_id} = \text{hash}(\text{column}) \% \text{numBuckets}$

Example:

department = IT
 $\text{hash}(\text{IT}) \% 4 = \text{bucket } 2$

→ goes to bucket_00002

Why partitions are NOT enough

You already ran:

```
SELECT*FROM employees_partWHERE department='IT';
```

✅ Fast (partition pruning)

But this query is slow:

```
SELECT*  
FROM employees_part e  
JOIN salaries s  
ON e.emp_id= s.emp_id;
```

Why?

- Spark must **shuffle both tables**

👉 Bucketing **avoids shuffle**

Why Bucketing exists (core reason)

| Bucketing aligns data so joins happen bucket-to-bucket, not cluster-wide

This is why Hive invented bucketing — **joins at scale**.

Bucketing Rules (IMPORTANT)

1. Bucketing works **only with Hive support**
2. Bucket column must be **same for both tables**

3. Number of buckets must be **same**
 4. Data must be **written using** `bucketBy()`
 5. Stored format must be **Parquet / ORC**
-

Your setup (what we'll do)

We'll:

1. Read your big `employees.csv`
 2. Create a **bucketed Hive table**
 3. Query it
 4. Verify buckets physically exist
-

Step 1: Spark Session (Hive enabled)

```
SparkSession spark = SparkSession.builder()
    .appName("BucketingExample")
    .master("local[*]")
    .enableHiveSupport()
    .config("spark.sql.warehouse.dir", "file:///C:/spark-warehouse")
    .getOrCreate();
```

Step 2: Read CSV

```
Dataset<Row> empDF = spark.read()
    .option("header", true)
    .option("inferSchema", true)
    .csv("src/main/resources/emp_big.csv");
```

Step 3: Create bucketed managed table

 **Bucketing happens ONLY at write time**

```
spark.sql("CREATE DATABASE IF NOT EXISTS company_db");
spark.sql("USE company_db");

spark.sql("DROP TABLE IF EXISTS employees_bucketed");

empDF.write()
  .format("parquet")
  .bucketBy(4,"department")// 👉 bucketing
  .sortBy("department")// optional but recommended
  .mode("overwrite")
  .saveAsTable("employees_bucketed");
```

What you'll see on disk

```
spark-warehouse/
├── company_db.db/
│   ├── employees_bucketed/
│   │   ├── part-00000-*.parquet
│   │   ├── part-00001-*.parquet
│   │   ├── part-00002-*.parquet
│   │   └── part-00003-*.parquet
```

 **Exactly 4 files = 4 buckets**

Step 4: Query the bucketed table

```
spark.sql(
  "SELECT department, COUNT(*) " +
```

```
"FROM employees_bucketed " +  
"GROUP BY department"  
).show(false);
```

Step 5: Prove bucketing is used (VERY IMPORTANT)

```
spark.sql(  
"EXPLAIN SELECT * FROM employees_bucketed WHERE department = 'IT'"  
).show(false);
```

You'll see:

```
Bucketed:true
```

Bucketing vs Partitioning (real-world rule)

Use Partitioning when:

- Filtering by date, country, department
- Data grows daily

Use Bucketing when:

- Frequent joins
- High cardinality columns
- Large tables on both sides

Use BOTH when:

- Huge data + frequent joins

```
.partitionBy("year")  
.bucketBy(8,"emp_id")
```

Common beginner mistakes (you avoided them)

- ✗ Thinking folders = buckets
- ✗ Expecting runtime bucketing
- ✗ Bucketing without Hive
- ✗ Different bucket count in join tables

Production truth (very important)

- Bucketing is **mostly a Hive-era optimization**
- Spark AQE (Adaptive Query Execution) reduced its necessity
- BUT:
 - Still used in **legacy Hive pipelines**
 - Still asked in **interviews**
 - Still useful for **very large joins**

You are now at solid intermediate Spark level

You now understand:

- RDD → DF → SQL
- Hive managed vs external
- Partition pruning
- Bucket pruning
- Physical vs logical data layout

Driver vs Executor

Rule #1 of Spark

Your Java code runs in the Driver.
Your data runs in the Executors.

This single sentence explains everything.

Where does your code run?

When you write:

```
Dataset<Row> rawDF = spark.read().csv(path);  
Dataset<Row> cleanDF = rawDF.withColumn(...);  
cleanDF.write().saveAsTable("customers");
```

All of that runs in the **Driver JVM**.

But the **rows** do NOT live there.

Let's see what really happens

Step 1 – You read CSV

```
spark.read().csv("customers-1000000.csv")
```

Driver:

- Reads file metadata
- Figures out file size

- Creates partitions
- Builds logical plan

Executors:

- Actually open the file
 - Read blocks
 - Parse rows
 - Create internal binary format
-

Step 2 – You convert dates

```
withColumn("SubscriptionDate", to_date(...))
```

Driver:

- Adds a transformation to the plan

Executors:

- For each row:
 - Read SubscriptionDate
 - Parse string
 - Convert to Date
 - Store result

This runs on **millions of rows**, so it MUST be executors.

Step 3 – You filter bad rows

```
filter(col("SubscriptionDate").isNotNull())
```

Driver:

- Adds filter to the plan

Executors:

- Evaluate condition on each row
 - Keep or drop row
-

Step 4 – Partition + Bucket

```
.partitionBy("Country")  
.bucketBy(8,"CustomerId")
```

Driver:

- Computes hash function
- Decides 8 buckets
- Decides folder structure

Executors:

- For every row:
 - Read Country
 - Hash CustomerId
 - Choose bucket
 - Write Parquet file
-

Step 5 – SQL Query

```
SELECT Country,COUNT(*)FROM customers_bucketedGROUPBY Country
```

Driver:

- Parses SQL
- Builds execution plan
- Chooses partition pruning
- Chooses shuffle strategy

Executors:

- Read Parquet files
- Only load needed columns
- Aggregate counts
- Send partial results back

Why RDD / DF / SQL feel different

API	What runs in Driver	What runs in Executors
RDD	Your lambdas	Every element
DataFrame	Column expressions	Every row
SQL	Query plan	Every row

RDD:

```
rdd.map(x → x *2)
```

The lambda runs **on executors**.

DataFrame:

```
withColumn("x2", col("x").multiply(2))
```

The expression runs **on executors**.

SQL:

```
SELECT x*2FROMtable
```

The multiplication runs **on executors**.

Only the **planning** is on driver.

The trap beginners fall into

This is wrong:

```
List<Row> rows = df.collectAsList();// ❌ pulls data into Driver
```

You just destroyed Spark.

You forced:

```
Executors → Driver RAM
```

Never do this on big data.

Job vs Stage Task

1 What is a Spark Job

A **job** is created whenever you run an **action**.

In your code:

```
result.show();
```


or

```
count();  
write();
```

These are **actions** → they trigger a **Spark Job**.

So:

```
result.show()  
→1 Spark Job
```

A job is:

“Everything needed to produce the final result”

2 What is a Stage

A job is split into **stages**.

A stage is:

“A group of tasks that can run without needing data from another partition”

Stages are created by **shuffle boundaries**.

Your query:

```
GROUPBY Country
```

requires a **shuffle**

Because all rows with the same Country must be brought together.

So Spark creates **2 stages**:

Stage 1:Read+map+extractCountry

Stage 2:Shuffle+aggregate

3 What is a Task

A **task** is:

| "One partition processed by one executor"

If Spark has:

200 partitions

Then:

Stage1 →200 tasks

Stage2 →200 tasks

Each task:

- Runs on one executor
- Handles one chunk of data

What happened in YOUR warehouse

When you queried:

```
SELECT Country,COUNT(*)FROM customers_bucketedGROUPBY Country;
```

Spark saw:

- Table is partitioned by Country
- So it can do **partition pruning**
- Each country folder becomes a task

Example:

```
Country=India → task 1  
Country=USA → task 2  
Country=Norway → task 3  
...
```

Stage 1:

```
ReadParquetfiles  
ExtractCountry  
Sendpartialcounts
```

Stage 2:

```
Shuffleby Country  
Combine counts  
Returnfinal results
```

Why bucketed tables are faster

If you join:

```
customers_bucketed  
JOIN orders_bucketed
```

ON CustomerId

Spark can do:

Bucket 1 ↔ Bucket 1
Bucket 2 ↔ Bucket 2
...

No shuffle → 1 stage instead of 2 → **huge speedup**

Real-life meaning

Term	What it really means
Job	One business question
Stage	One phase of data movement
Task	One CPU working on one data chunk

You've now built and run **true distributed analytics**, just on a laptop 🤖